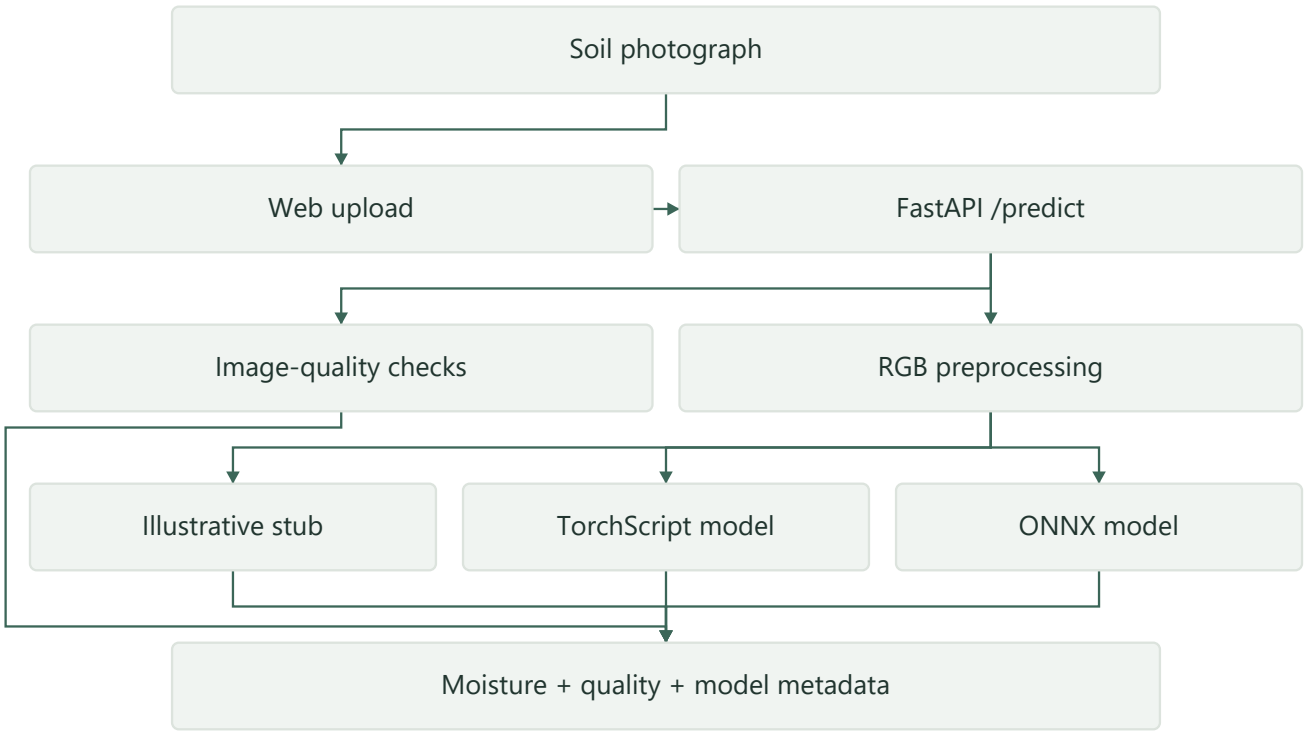
AgroDry (Stubby)

Image-based soil-moisture estimation

An image-upload interface and inference API for soil-moisture estimation, supported by preprocessing and frequency-aware training experiments.

Architecture



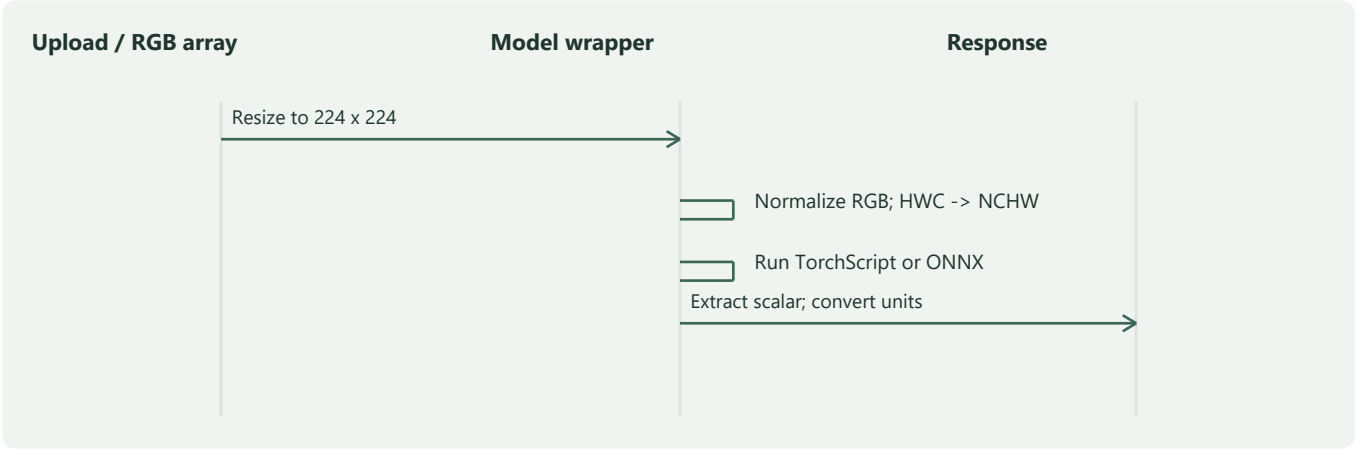
Component overview derived from the supplied source; version-specific details may differ.

Component	Implementation	Role
Interface	HTML image-upload pages	Image selection and returned estimate
API	FastAPI routes	Health, version, prediction
Quality checks	Brightness + sharpness	Heuristic image feedback
Model wrapper	Stub / TorchScript / ONNX	Configurable inference path
Training	ResNet + AFSC/DCT experiments	Model development material

Application prototype

Inference contract & tensor path

AgroDry (Stubby) is the application boundary around a soil-moisture estimator. Its central engineering contract translates an uploaded image into a consistent numerical tensor and then translates an exported model scalar into a response. Training code, the exported model and the serving wrapper must agree on that contract.



Input transformation

The wrapper converts uint8 RGB to float32 in [0,1], subtracts channel means (0.485, 0.456, 0.406), and divides by standard deviations (0.229, 0.224, 0.225). It then transposes and batches the array to shape 1 x 3 x 224 x 224. OpenCV bilinear resizing is used when available, with a PIL fallback; export parity should account for this implementation choice.

Output interpretation

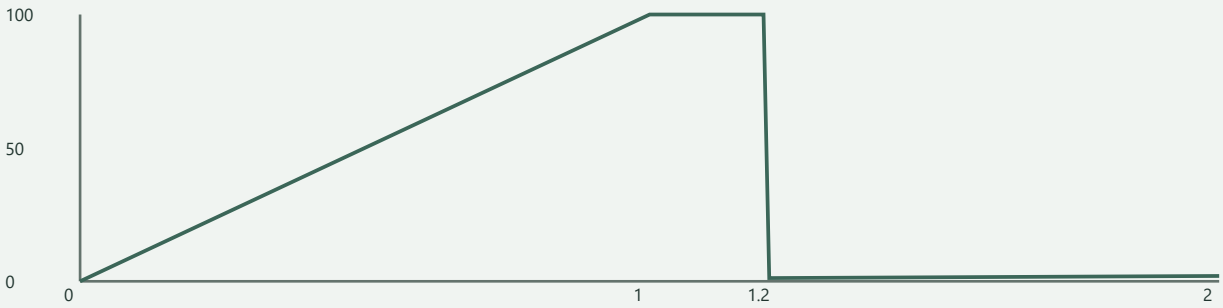
TorchScript accepts a tensor or a dictionary with a moisture entry; ONNX reads its first input and first output. Both flatten the selected output and take its first value. This makes output order, name and shape important export assumptions. The moisture response contains value_pct and confidence, while separate fields report model version, backend, latency and image-quality notes.

Stage	Contract	Potential mismatch
Export	One intended moisture scalar	Multi-output exports can select the wrong result
Normalization	ImageNet RGB mean / standard deviation	Trainer may use different scaling
Units	Fraction or percentage	An automatic numeric-range heuristic is ambiguous

Source / interpretation: AI Moisture Detection/Backend/model.py, schemas.py, utils.py and app.py; selected source snapshots accompany the website.

Backend selection & score semantics

The serving code supports three modes. The stub is useful for interface development, while TorchScript and ONNX require matching exported weights and installed runtimes. A technical demonstration must identify the actual execution branch, because the configured backend label alone does not prove that a trained model ran.



Raw output: 0 to 2. Response percentage: 0 to 100. Drop occurs above 1.2.

Configured mode versus actual mode

Initialization loads TorchScript only when the mode, model path and framework availability agree; ONNX follows an equivalent gate. Otherwise prediction falls through to the stub. A load failure inside an attempted real-model load can instead raise an error. Prediction metadata still uses the configured backend, so some fallback responses can appear to identify a real model even when no session was loaded.

The percentage discontinuity

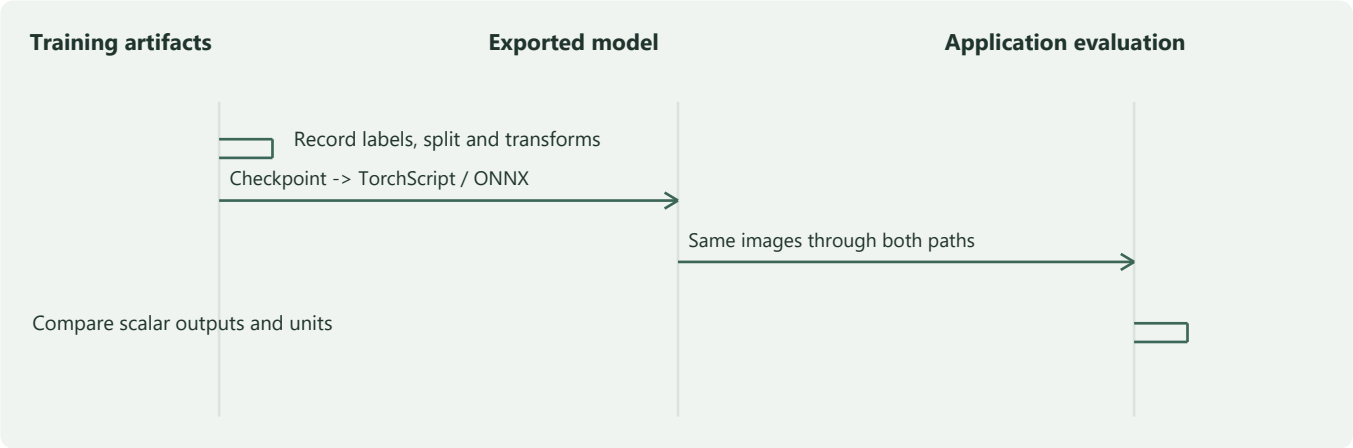
The source multiplies raw outputs between 0 and 1.2 by 100 and clips to [0,100]; values outside that range are treated directly as percentages and clipped. An illustrative output of 1.2 therefore becomes 100%, while 1.21 becomes 1.21%. The plot shows the code mapping, not soil measurements. A versioned export should declare units explicitly instead of guessing them from magnitude.

Field / mode	Actual source behavior
Stub moisture	Random value around 24%, ranging approximately 21-27%; independent of image content
Confidence	Fixed at 0.90 in stub and real-output postprocessing; not estimated uncertainty
Deployment readiness	Expose actual backend; bind model hash and units; verify image decoding and export equivalence

Source / interpretation: AI Moisture Detection/Backend/model.py: QFMModel.__init__, predict, _post_moisture and _predict_stub. No model weights were executed for this analysis.

Training-to-serving evidence

The application contains earlier training experiments and AFSC-related research code as well as the API. Connecting these layers requires an explicit model lineage: dataset and split, training configuration, checkpoint, export, serving wrapper and evaluation. A directory name or QFM label is not enough to establish that lineage.



Adaptive frequency training

The AFSC trainer describes low-frequency DCT masks in ResNet blocks, warm-started weights, mask regularization, mixed precision and TensorBoard visualization. These mechanisms can be documented as source-level architecture, but the exact exported model used by the app remains to be matched to its checkpoint and configuration. Earlier Draft1 trainers should retain separate identities.

A useful evaluation protocol

A reproducible release would first compare native and exported predictions on the same held-out images, then report moisture error in the datasetâ€™s declared units. Grouping splits by collection session, field or sample can help expose capture leakage. Lighting and sharpness diagnostics should be studied as input-quality indicators rather than called confidence estimates. These are proposed checks; no accuracy result is supplied here.

Artifact	What it should establish
Label definition	Physical unit, acquisition procedure and target normalization
Split manifest	Which samples trained, selected and tested the model
Deployment trace	Model hash, actual execution backend, output schema and latency setup

Source / interpretation: AI Moisture Detection/afsccnn, Draft1 and Data; backend modules. Relationship to WERM and QFM remains a project connection rather than a shared accuracy claim.

Evidence & source

Implementation, supporting records, and the boundary of the claim

Area	Evidence	Reference
Implemented surface	Upload interface and inference wrapper	Local backend and frontend source
Execution modes	Three configured backend options	Backend/model.py
Current result boundary	No field-accuracy claim added	Trained export and evaluation to be reconciled

Selected implementation files

File / record	Path / reference
model.py	AI Moisture Detection/Backend/model.py
schemas.py	AI Moisture Detection/Backend/schemas.py
utils.py	AI Moisture Detection/Backend/utils.py

Selected inference and schema modules. Model weights and the full application release remain separate artifacts.

Evidence boundary

The default inference mode is an illustrative stub. A public trained-model demonstration requires the correct export, backend configuration, and evaluation record. A displayed percentage is not evidence of physical water-content calibration.

Technical note

The wrapper resizes images to 224 × 224 and uses normalized RGB tensors. It can fall back to the stub if a requested trained backend is unavailable; actual execution mode must be made explicit.